PYTHON SUMMER PLAN · DAY 03

Inheritance & Composition

Build a Shape hierarchy. Reuse code via is-a (inheritance) and has-a (composition).

Wednesday, May 13 · 75 minutes

Yuvraj Ruparel

R E C A P · Y E S T E R D A Y

Where we left off

Vector3D has __add__, __sub__, __mul__, __rmul__, __eq__, __abs__, __len__, plus a magnitude @property. It behaves like a built-in type. Today the question shifts: how do we build a family of related classes that share behavior without copy-pasting it?

T H E P R O B L E M

Copy-paste everywhere

```
class Circle:
    def __init__(self, x, y, r):
        self.x, self.y = x, y      # repeated
        self.r = r

class Box:
    def __init__(self, x, y, w, h):
        self.x, self.y = x, y      # same lines
```

T H E F I X

Hierarchy + composed state

```
class Shape:
    def __init__(self, pos, vel):
        self.pos = pos    # Vector3D
        self.vel = vel

class Circle(Shape):
    ...
class Box(Shape):
    ...
```

Same idea, half the code. Inheritance for what they share, composition for what they hold.

0 1 · CONCEPT

is-a vs has-a

Two different relationships. Pick the right one and the class design writes itself.

I S - A (Inheritance)

Subclass extends parent

A Circle is a Shape. A Box is a Shape. They all need position, velocity, update().

```
class Circle(Shape):
    def __init__(self, pos, vel, radius):
        super().__init__(pos, vel)
        self.radius = radius
```

H A S - A (Composition)

Class holds another object

A Shape has a position (Vector3D) and a velocity (Vector3D). The vector class does the math.

```
class Shape:
    def __init__(self, pos, vel):
        self.pos = pos      # Vector3D
        self.vel = vel      # Vector3D
```

Rule of thumb: is-a → inheritance. has-a → composition. Mix both freely.

Shape — the parent

```python
from vector3d import Vector3D

class Shape:
    def __init__(self, pos, vel):
        self.pos = pos
        self.vel = vel

    def update(self, dt, g=9.81):
        self.vel = self.vel + Vector3D(0, -g, 0) * dt
        self.pos = self.pos + self.vel * dt

    def __repr__(self):
        name = type(self).__name__
        return f"{name}(pos={self.pos}, vel={self.vel})"
```

Three things to notice

- **Composition.**

  pos and vel are Vector3D instances. Shape doesn't know vector math — it asks Vector3D for it.

- **update(dt).**

  Symplectic Euler step. Gravity edits velocity, velocity edits position. Pure model — no drawing here.

- **__repr__ via type(self).**

  Subclasses get accurate names — Circle(pos=…), Box(pos=…). No override needed.

Circle(Shape) — extending the base

```
class Circle(Shape):
    def __init__(self, pos, vel, radius):
        super().__init__(pos, vel)        # reuse parent
        self.radius = radius

    @property
    def area(self):
        return math.pi * self.radius ** 2

    def collides_with(self, other):
        d = abs(self.pos - other.pos)
        return d < self.radius + other.radius
```

OUTPUT

```
c = Circle(Vector3D(0,5,0), Vector3D(2,0,0), radius=0.5)
c.update(0.1)
print(c)
Circle(pos=Vector3D(0.2, 4.9, 0), vel=Vector3D(2, -0.98, 0))
```

What super() does

super().__init__(pos, vel) calls Shape.__init__ on the same instance. Circle gets pos and vel set up for free — then adds radius on top.

Subclass adds two things

1. radius — Circle-specific state.
2. collides_with — Circle-specific behavior (distance < sum of radii).

Box(Shape) — different geometry

```
class Box(Shape):
    def __init__(self, pos, vel, width, height):
        super().__init__(pos, vel)
        self.width  = width
        self.height = height

    @property
    def area(self):
        return self.width * self.height

    def collides_with(self, other):  # AABB
        dx = abs(self.pos.x - other.pos.x)
        dy = abs(self.pos.y - other.pos.y)
        return (dx < (self.width + other.width)/2
                and dy < (self.height + other.height)/2)
```

Same shape of code,

different math.

Circle and Box both inherit update() and __repr__ from Shape. Both override area and collides_with because the geometry differs.

AABB = Axis-Aligned Bounding Box.

Centers within half-width on both axes → overlap.

Override what changes, inherit what doesn't

METHOD	DEFINED IN	BEHAVIOR
update(dt)	Shape	*Inherited unchanged. Same physics for every subclass.*
__repr__	Shape	*Inherited. type(self).__name__ → "Circle(...)" or "Box(...)"*
area	Circle, Box	*Overridden. πr^2 vs $w \cdot h$. Same name, different math.*
collides_with	Circle, Box	*Overridden. Distance check vs AABB.*
__init__	Shape + each	*Each subclass calls super().__init__ then adds its own attributes.*

Treat them all as Shape

One list, mixed types. The simulation loop never checks what kind of shape it is — it just calls update() and collides_with().

```
shapes = [
    Circle(Vector3D(-2, 3, 0), Vector3D(1, 0, 0), radius=0.5),
    Box(Vector3D(2, 4, 0), Vector3D(-1, 0, 0), width=1.0, height=1.0),
    Circle(Vector3D(0, 6, 0), Vector3D(0, 0, 0), radius=0.3),
]

for step in range(200):
    for s in shapes:
        s.update(dt=0.02)          # same call works for every subclass
    for a, b in itertools.combinations(shapes, 2):
        if a.collides_with(b): resolve(a, b)
```

Takeaway

This is polymorphism. The loop doesn't care which subclass it has — Python dispatches the right method at call time. Add a Triangle(Shape) tomorrow and the loop still works without a single edit.

TODAY'S EXERCISE · 75 MIN

2D Shape Physics Sandbox

Mixed circles and boxes bouncing inside a container. Gravity pulls down. Shapes collide elastically. Walls bounce. matplotlib animation only — pygame is Week 2.

SPEC

Module structure

shapes.py with Shape, Circle, Box.

Reuse Vector3D from yesterday — do not rewrite vector math.

Required classes

Shape (pos, vel, update, __repr__)

Circle(Shape) (radius, area, collides_with)

Box(Shape) (w, h, area, AABB collides_with)

Behavior

Gravity (g = 9.81, downward)

Elastic shape-shape collisions

Walls bounce (flip relevant velocity component)

matplotlib FuncAnimation

Tips & gotchas

Always call super().__init__

Skip it and pos/vel never get set. Circle blows up the first time you call .update() — AttributeError on self.pos.

Don't copy Vector3D fields into Shape

Hold the Vector3D object. Don't write self.x, self.y, self.z. That defeats composition and the two copies drift out of sync.

Override only what differs

area and collides_with change per shape. update() and __repr__ don't — leave them on the parent. Less code, fewer bugs.

Elastic collision = swap normal components

Circle-circle: project velocities onto the line connecting centers, swap. Walls: flip vx or vy on contact. Start with walls — collision response is the tricky part.

W R A P - U P · W H A T T H I S F E E D S

shapes.py becomes a building block

The classes you write today aren't a one-day exercise. They get reused all week.

T H U R S D A Y

Validation

Add @property setters: negative width / height / radius raise ValueError. Box.square(s) and Circle.unit() as @classmethod factories.

F R I D A Y

Exceptions

Wrap collision math in try/except. Build custom exception classes. Fix the normalize() zero-magnitude bug in vector3d.py.

S A T U R D A Y

Spring-Mass-Damper

The mass gets drawn as a Box from this same module. Same import, same class. Reuse, not rewrite.

When the animation runs cleanly, drop the file back here.